



Testing

▼ Testing en el Proceso Unificado de Desarrollo (PUD)

Etapas del PUD

1. **Requerimientos** (objetivos y verificables)
2. **Diseño**
3. **Implementación**
4. **Pruebas**
5. **Despliegue**

En el **Proceso Unificado**, el testing se ubica en el **penúltimo flujo de trabajo**, dentro de la fase de **Pruebas**.

▼ Conceptos Fundamentales del Testing

Un **testing exitoso** es aquel que **detecta defectos**. No se puede garantizar que no haya defectos con el testing: **no asegura calidad**, simplemente **evidencia los defectos del software**.

El **Agilismo** intenta romper con el concepto de que el testing es una actividad destructiva o una crítica al trabajo de los demás. En realidad, el testing es una **actividad de control sobre los artefactos creados por otros**.

Las **competencias de los testers y de los desarrolladores** son distintas: los testers suelen tener un **mayor nivel de detalle** y una **mirada más analítica**.

En el **PUD**, el testing se realizaba casi al final porque se creía que **no se podía pensar en pruebas sin antes tener código implementado**. En cambio, en las **metodologías ágiles**, el testing **se hace desde el principio**.

▼ Requerimientos Verificables

Si un requerimiento no se puede probar, es decir, no es verificable, entonces no se puede construir ni verificar.

Esto se relaciona con la **última "C" de las User Stories: Confirmation**, que implica que cada historia debe tener **criterios de aceptación claros y comprobables**.

▼ Testing en el marco del PUD

El **testing es la actividad más costosa** del proceso: representa entre el **30 % y, en software críticos, hasta el 50 % del costo total** del desarrollo.

Por eso, muchas empresas lo consideran un gasto, cuando en realidad es una inversión para prevenir errores más caros.

Los costos del testing están directamente relacionados con el **esfuerzo humano** necesario para realizarlo.

Además, **cuanto más exitoso es el desarrollo, más difícil es que el testing encuentre defectos**.

Los defectos, generalmente, **provienen de etapas anteriores** como requerimientos, análisis o diseño.

▼ Error vs Defecto vs Falla

- **Error:** cuando el problema **no se traslada a otra etapa**.

Ejemplo: si corrijo un requerimiento mal redactado durante la etapa de requerimientos, el error **no pasa a la siguiente fase**.

- **Defecto:** cuando un error **se detecta en una etapa posterior**.

Ejemplo: si encuentro un problema en implementación que proviene de análisis o diseño, eso es un **defecto**.

- **Falla:** es el **mal funcionamiento del sistema** (por ejemplo, que se trabe o se cierre). A veces puede ser subjetiva según la percepción del cliente.

💡 Ejemplo: un error ortográfico es un error, si lo detecta testing pasa a ser un defecto, aunque no genere una falla real en el sistema.

Los **errores más caros** son los de **requerimientos**, luego los de **arquitectura** y finalmente los de **programación**.

Hoy en día, quienes realizan testing deben saber algo de **implementación** para poder usar **herramientas de automatización**.

El testing, en definitiva, es una **actividad de control**, en la que se compara una versión del software con los **requerimientos documentados** (por ejemplo, *user stories*).

▼ Testing Metodológico vs Testing Ad Hoc

◆ Testing Ad Hoc

El **testing ad hoc** consiste en “tocar hasta que aparezca el error”, sin planificación previa.

El problema es que luego **no se recuerdan los pasos realizados**, por lo tanto, **no se puede reproducir el error** ni reportarlo correctamente al desarrollador.

Es un **testing irreproducible** y poco útil para el proceso de corrección.

◆ Testing Metodológico

En cambio, el **testing metodológico** se basa en un **proceso estructurado**, con etapas y artefactos definidos:

1. Planificación y Diseño

- Se genera un **Plan de Prueba** y **Casos de Prueba**.
- Un **caso de prueba** describe paso a paso un escenario específico, con un seteo determinado (datos concretos) y un **resultado esperado**.
- Cuanto más detallado sea, más fácil es **reproducir el error**.

💡 Ejemplo:

El sistema pide país y el usuario selecciona *Israel*. El resultado esperado es que aparezcan los datos correspondientes a Israel.

Idealmente, este nivel de detalle debería generarse desde la **etapa de requerimientos** (para adelantar trabajo).

Así, en la fase de testing solo se ejecutan los pasos de **Ejecución y Seguimiento**, porque ya está definido qué se espera obtener.

2. Ejecución

- Se realizan las pruebas y se documentan los resultados.
- Se genera un **Reporte de Defectos** (por ejemplo, en **Jira** o **Trello**), donde se puede hacer el **seguimiento** del error.

3. Seguimiento y Cierre

- El reporte pasa a desarrollo para su corrección y vuelve a testing para su verificación.
- Este proceso forma un **Ciclo de Prueba**:
 - El primer ciclo se denomina **Ciclo 0**.
 - Luego siguen **Ciclo 1, 2, ...**, hasta que el sistema sale a producción.
- Puede liberarse con errores leves, **nunca con errores bloqueantes**.
- Cada ciclo tiene **un costo** asociado.

▼ Tareas del Testing

Los **casos de prueba** deben compararse con los **requerimientos**, y a partir de eso se pueden realizar dos actividades fundamentales:

- **Validación**: comprobar que se construyó lo que el usuario espera (*adecuación*).
- **Verificación**: comprobar que el sistema funciona correctamente (*corrección*).

💬 El desarrollo y el testing son procesos inversos:

- El desarrollo va **de lo general a lo particular**.
- El testing va **de lo particular a lo general**.

▼ Niveles de Prueba

Nivel	Responsable	Descripción
Unitario	Desarrollador	Prueba individual de cada módulo o función.
Integración	Testing / Dev	Se realiza en un ambiente separado (ambiente de prueba). En la práctica, los desarrolladores hacen parte de esta etapa antes de pasar a testing.
Sistema	Testing	Evalúa el sistema completo como un todo.
UAT (User Acceptance Test)	Product Owner	Se realiza en preproducción o, si no existe, en producción , con las precauciones necesarias.

▼ Ambientes y Entornos 🌐

1. **Desarrollo:** entorno del programador.
2. **Prueba:** ambiente aislado sin base de datos real; si se modifica, debe reiniciarse el ciclo.
3. **Preproducción:** replica del entorno real (muchas empresas no lo tienen por costo).
4. **Producción:** el entorno real.

⚠ Ejemplo: si no hay preproducción, el sistema sale directamente "a la vida real", y pueden ocurrir errores graves (como emitir mal tarjetas de crédito).

▼ Pruebas con y sin Regresión ↺

- **Con regresión:** se vuelven a correr **todas las pruebas desde el ciclo 0** cada vez que se corrige algo.
 - Es la opción más segura, pero también la más costosa.
 - Generalmente se hace con **testing automatizado**.
- **Sin regresión:** se continúa desde donde se dejó luego de la corrección.
 - Es más rápido, pero más riesgoso.

💡 "Cuando se corrige un error, suelen aparecer tres nuevos."

▼ Tipos de Prueba

Dependen de si los **requerimientos son funcionales o no funcionales**.

- **Smoke Test:** se hace antes de comenzar los ciclos formales, para verificar que el sistema básico funciona.
- **Pruebas exploratorias:** se realizan para **ganar conocimiento del sistema**, especialmente cuando no hay mucha documentación o el tester no participó en las fases de requerimientos o diseño.
 - Su objetivo no es encontrar defectos, sino **conocer el sistema**.

▼ Severidad y Prioridad

La **severidad** se acuerda dentro de cada empresa y clasifica los errores.

La **prioridad**, en cambio, la establece el **cliente** y no siempre se relaciona directamente con la severidad.

Niveles de Severidad

- **Bloqueante:** impide el uso del sistema.
- **Grave:** el sistema funciona, pero con errores importantes (por ejemplo, debía multiplicar y divide).
- **Leve:** afecta aspectos menores, como ingresar decimales.
- **Cosmética:** cuestiones visuales o de presentación (colores, tamaños, ortografía).

Niveles de Prioridad

- **Urgente**
- **Media**
- **Baja**

💡 Ejemplo: un error leve puede tener prioridad urgente si afecta a un cliente clave o una operación crítica.

El equipo de desarrollo decide **en qué orden corregir** los errores.

El equipo de testing **cierra el error** cuando verifica que fue resuelto correctamente.